Generic Launch/Takeoff Pitch Acceptance State Machine for ArduPlane

Implement a configurable launch/takeoff state machine for ArduPlane that monitors actual pitch angle during takeoff and completes the launch phase only when the aircraft achieves and maintains a desired pitch angle within a specified tolerance for a required duration before handing control over to normal waypoint navigation.

---

1. Source Tree Analysis & Architecture Inspection

Following the inspection of the current ArduPilot codebase:

A. Where fixed-wing takeoff is controlled

- **Mission Takeoff (`AUTO` mode, `MAV_CMD_NAV_TAKEOFF`)**:
- Initiated in Plane::do_takeoff().
- Actively controlled in takeoff.cpp:
- Plane::takeoff_calc_roll(): Maintains level wings or heading hold close to ground.
- Plane::takeoff_calc_pitch(): Manages ground roll pitch (TKOFF_GND_PITCH), rotation speed (TKOFF_ROTATE_SPD), and takeoff climb pitch target (auto_state.takeoff_pitch_cd).
- Plane::takeoff_calc_throttle(): Controls takeoff throttle slew, max throttle, and hand/catapult launch detection.
- **Dedicated Takeoff Mode (`TAKEOFF`)**:
- Handled in ArduPlane/mode_takeoff.cpp (ModeTakeoff), which climbs towards TKOFF_ALT at heading TKOFF_DIST.

B. Where takeoff completion is determined

- **In `AUTO` mode**:
- AP_Mission periodically queries Plane::verify_command().

- For MAV_CMD_NAV_TAKEOFF, this delegates directly to Plane::verify_takeoff().
- Currently, verify_takeoff() checks:

```cpp
int32_t relative_alt_cm = adjusted_relative_altitude_cm();
if (relative_alt_cm > auto_state.takeoff_altitude_rel_cm ||
plane.check_takeoff_timeout_level_off()) {
auto_state.takeoff_complete = true;
...
return true;
}
```

- When verify_takeoff() returns true, the mission advances to the next waypoint (typically MAV_CMD_NAV_WAYPOINT), handing control over to normal waypoint navigation.
- **In TAKEOFF mode**:
- Evaluated in ModeTakeoff::update(), switching flight_stage from TAKEOFF to NORMAL once altitude or distance is satisfied.

C. Where current pitch is obtained

- Obtained from ArduPilot's AHRS attitude estimator via ahrs.pitch_sensor.
- **Units**: ahrs.pitch_sensor is in **centidegrees** (int32_t), where $1^\circ = 100\text{ cd}$ (e.g. $15.0^\circ = 1500\text{ cd}$, $-5.0^\circ = -500\text{ cd}$).
- In degrees (float): ahrs.pitch_sensor * 0.01f or degrees(ahrs.get_pitch()).
- To prevent unit bugs, all user parameters will be cleanly converted between degrees (UI) and centidegrees (internal math).

D. Where QuadPlane takeoff/transition is controlled

- MAV_CMD_NAV_VTOL_TAKEOFF is initiated in QuadPlane::do_vtol_takeoff() and verified in QuadPlane::verify_vtol_takeoff().
- Vertical climb completes when plane.current_loc.alt >= plane.next_WP_loc.alt.

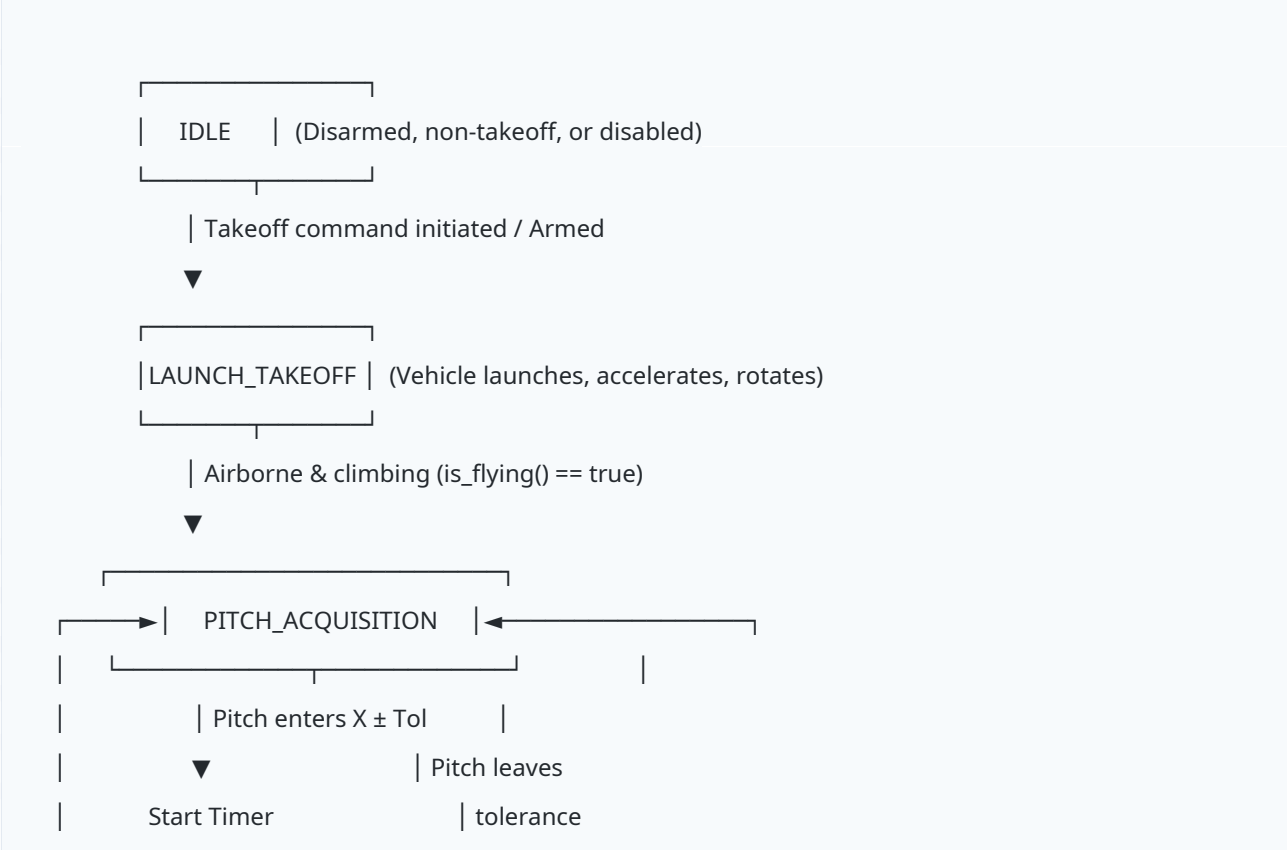
- Then forward transition to fixed-wing is driven by SLT_Transition / Tailsitter_Transition / Tiltrotor_Transition through states AIRSPEED_WAIT $\rightarrow$ TIMER $\rightarrow$ DONE.
- If a QuadPlane uses a runway launch (MAV_CMD_NAV_TAKEOFF), it routes through Plane::verify_takeoff().

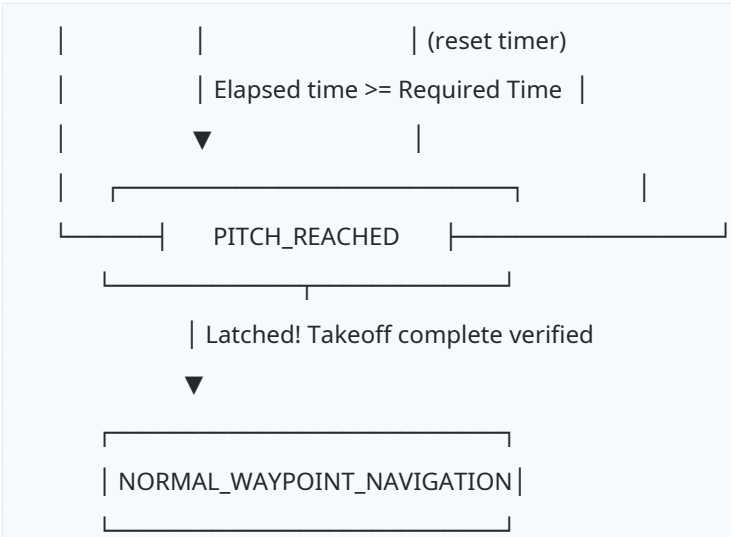
E. The safest place to add the new condition

- Create a dedicated, modular LaunchStateMachine helper inside ArduPlane/takeoff.cpp and declared in ArduPlane/Plane.h.
- Gate takeoff completion in Plane::verify_takeoff().
- If the launch pitch condition is enabled: takeoff completion requires launch_state_machine.is_pitch_reached() (while airborne).
- If disabled: falls back 100% to existing ArduPilot behavior.
- Hook into Plane::do_takeoff(), QuadPlane::do_vtol_takeoff(), and ModeTakeoff::_enter() to reset state at the beginning of each launch sequence.

---

2. Proposed Architecture & State Machine





State Machine Implementation

The state machine will be encapsulated in `LaunchStateMachine`:

```
class LaunchStateMachine {
public:
    enum class State : uint8_t {
        IDLE = 0,
        LAUNCH_TAKEOFF = 1,
        PITCH_ACQUISITION = 2,
        PITCH_REACHED = 3,
        NORMAL_NAVIGATION = 4,
    };

    void reset();
    void start();
    bool update();
    bool is_pitch_reached() const;
    State get_state() const;

private:
    State state = State::IDLE;
    uint32_t stability_start_ms = 0;
    uint32_t last_debug_ms = 0;
    bool latched = false;
};
```

---

3. Parameter Design

> **NOTE:** ArduPilot strictly enforces `AP_MAX_NAME_SIZE = 16` on all parameter full names. A 17-character name like `LAUNCH_PITCH_TIME` would be rejected at runtime. We use `LAUNCH_PITCH_SEC` (16 chars), adhering to standard ArduPilot convention (e.g., `TKOFF_PLIM_SEC`).

The parameters will be added to `ParametersG2` `g2` in ArduPlane/Parameters.h and ArduPlane/Parameters.cpp:

Parameter	Type	Default	Range / Units	Description
LAUNCH_PITCH_EN	AP_Int8	0	0:Disabled, 1:Enabled	Master enable switch for pitch-based launch state machine
LAUNCH_PITCH	AP_Float	15.0f	-45.0 to 85.0 deg	Desired pitch angle for acceptance condition
LAUNCH_PITCH_TOL	AP_Float	2.0f	0.1 to 15.0 deg	Acceptance tolerance around desired pitch angle
LAUNCH_PITCH_SEC	AP_Float	1.0f	0.0 to 30.0 s	Continuous duration pitch must remain within tolerance

---

4. Proposed Changes

[Component: ArduPlane Core]

[MODIFY] [`ArduPlane/Parameters.h`](file:///home/amaan/Work/BSVT/ardupilot/ArduPlane/Parameters.h)

- Add parameter declarations in `ParametersG2`:
- `AP_Int8 launch_pitch_enable;`
- `AP_Float launch_pitch;`
- `AP_Float launch_pitch_tol;`

- `AP_Float launch_pitch_time;`

[MODIFY] [`ArduPlane/Parameters.cpp`](file:///home/amaan/Work/BSVT/ardupilot/ArduPlane/Parameters.cpp)

- Add `LAUNCH_PITCH_EN`, `LAUNCH_PITCH`, `LAUNCH_PITCH_TOL`, `LAUNCH_PITCH_SEC` definitions in `ParametersG2::var_info[]` with indices 42-45.

[MODIFY] [`ArduPlane/Plane.h`](file:///home/amaan/Work/BSVT/ardupilot/ArduPlane/Plane.h)

- Declare `class LaunchStateMachine` and embed member instance `LaunchStateMachine launch_sm;` on `Plane`.
- Add getter methods and helper functions.

[MODIFY] [`ArduPlane/takeoff.cpp`](file:///home/amaan/Work/BSVT/ardupilot/ArduPlane/takeoff.cpp)

- Implement `LaunchStateMachine::reset()`, `start()`, `update()`.
- Implement stability timer logic and edge-triggered state latching.
- Add rate-limited (1 Hz) telemetry output for debug tracking (`gcs().send_text()`).
- In `Plane::auto_takeoff_check()` or disarm: trigger `launch_sm.reset()`.

[MODIFY] [`ArduPlane/commands_logic.cpp`](file:///home/amaan/Work/BSVT/ardupilot/ArduPlane/commands_logic.cpp)

- In `Plane::do_takeoff()`:
- If `g2.launch_pitch_enable`, call `launch_sm.start()`.
- If mission takeoff command pitch `cmd.p1 <= 0` and `g2.launch_pitch_enable`, set target climb pitch to `g2.launch_pitch`.
- In `Plane::verify_takeoff()`:
- If `g2.launch_pitch_enable`:
- Call `launch_sm.update()`.
- Check `launch_sm.is_pitch_reached()`.
- When satisfied and `is_flying()`, complete takeoff and set state to `NORMAL_NAVIGATION`.

[MODIFY] [`ArduPlane/quadplane.cpp`](file:///home/amaan/Work/BSVT/ardupilot/ArduPlane/quadplane.cpp)

- In `QuadPlane::do_vtol_takeoff()`: trigger `plane.launch_sm.reset()` / `start()`.
- In `QuadPlane::verify_vtol_takeoff()`: integrate pitch monitoring after vertical climb completes during forward transition.

---

5. Verification Plan

Automated & Unit Tests

1. **Compilation**:

`./waf configure --board sitl && ./waf plane`

Ensure 0 warnings and clean binary output.

1. **Unit Test Scenarios (via Python SITL test runner or scripted SITL)**:

- **Test A: Pitch never reached** (`LAUNCH_PITCH = 45`, vehicle climbs at standard 15°):

Verify `PITCH_REACHED` remains `false`, takeoff does not complete.

- **Test B: Briefly crosses angle** (`LAUNCH_PITCH = 15`, `LAUNCH_PITCH_SEC = 2.0`, rapid pitch variation):

Verify condition does not latch prematurely.

- **Test C: Stable at desired angle** (`LAUNCH_PITCH = 15`, `LAUNCH_PITCH_TOL = 2.0`, `LAUNCH_PITCH_SEC = 1.0`):

Verify stability timer reaches 1.0s, `PITCH_REACHED` latches, and waypoint 2 is reached.

- **Test D: Enters range then leaves**:

Verify timer resets to 0 when leaving tolerance window.

- **Test E: Negative pitch angle**:

Verify signed math works correctly for negative angles.

Manual SITL Verification

1. Start SITL simulator:

`Tools/autotest/sim_vehicle.py -v ArduPlane --console --map`

1. Configure parameters:

```text
param set LAUNCH_PITCH_EN 1
param set LAUNCH_PITCH 15.0
param set LAUNCH_PITCH_TOL 2.0
param set LAUNCH_PITCH_SEC 1.0
```

1. Load waypoint mission (Takeoff $\rightarrow$ Waypoint 1 $\rightarrow$ Waypoint 2).

1. Run mission:

`mode AUTO`

`arm throttle`

1. Observe MAVProxy/Mission Planner console output:

- Debug message reporting current pitch, tolerance status, and elapsed stable time.
- Notice message: `Launch: PITCH_REACHED!`.
- Vehicle transitioning to normal waypoint navigation.